

XSS (Cross-Site Scripting)

XSS — атака, при которой злоумышленник внедряет вредоносный JavaScript-код в страницу через пользовательский ввод. Если приложение выводит данные без экранирования, скрипт выполнится в браузере другого пользователя и может похитить cookie, токены сессии или перенаправить на фишинговый сайт.

Применённые методы защиты

Во всех PHP-файлах (form.php, edit.php, login.php, admin.php) при выводе любых пользовательских данных используется функция htmlspecialchars() с параметрами по умолчанию. Это преобразует специальные символы HTML, в сущности, предотвращая выполнение скриптов.

Если пользователь введёт `<script>alert(1)</script>`, в HTML будет выведено `<script>alert(1)</script>`, и скрипт не выполнится.

```
<input type="text" name="name" value="<?= htmlspecialchars($values['name'] ?? '') ?>">
<?php if (isset($errors['name'])): ?>
    <div class="error-msg"><?= htmlspecialchars($errors['name']) ?></div>
<?php endif; ?>
```

Рис 1. htmlspecialchars().

Вывод: уязвимость XSS отсутствует.

Information Disclosure (Утечка информации)

Описание уязвимости

Information Disclosure — утечка внутренней информации о системе (детали ошибок БД, пути к файлам, версии библиотек, логины/пароли в исходном коде). Это может помочь злоумышленнику подобрать дальнейший вектор атаки.

Применённые методы защиты

Все ошибки логируются на сервере через `error_log()`, пользователю возвращается только общее сообщение («Internal server error. Try again later»).

```
if ($appId === false) {  
    http_response_code(500);  
    echo 'Internal server error. Try again later';  
    exit;  
}
```

Рис 2. Обработка ошибок в бд.

```
catch (PDOException $e) {  
    $pdo->rollBack();  
    error_log('saveToDatabase error: ' . $e->getMessage());  
    return false;  
}
```

Рис 3. Запись в лог.

Учётные данные БД и JWT-секрет читаются из переменных окружения (`getenv()`), а не хранятся в коде.

```
$host = getenv('DB_HOST') ?: 'localhost';  
$dbname = getenv('DB_NAME') ?: 'u82356';  
$username = getenv('DB_USER') ?: 'u82356';  
$password = getenv('DB_PASS') ?: '9869396';
```

Рис 4. Переменные окружения.

```
$secret = getenv('JWT_SECRET');  
if ($secret === false) {  
    http_response_code(500);  
    die('JWT_SECRET environment variable is not set');  
}
```

Рис 5. Наличие jwt secret.

Вывод: риск Information Disclosure минимален

SQL Injection

Описание уязвимости

SQL Injection — атака, при которой злоумышленник внедряет произвольный SQL-код через пользовательский ввод. При конкатенации строк в запросе ввод пользователя становится частью SQL и может изменить логику запроса, получить доступ к чужим данным или уничтожить таблицы.

Применённые методы защиты

Во всём приложении используются исключительно подготовленные запросы (PDO prepared statements) с плейсхолдерами (:name, ?). Данные пользователя никогда не попадают в текст SQL запроса — они передаются отдельно и автоматически экранируются драйвером PDO.

Дополнительно используются транзакции для атомарности операций (вставка заявки и языков, обновление и удаление). Это гарантирует целостность данных и предотвращает частичные изменения.

```
$pdo->beginTransaction();
$stmt = $pdo->prepare("
    INSERT INTO applications (full_name, phone, email, birth_date, gender, biography, contract_accepted)
    VALUES (:full_name, :phone, :email, :birth_date, :gender, :biography, 1)
");
$stmt->execute([
    ':full_name' => $data['name'],
    ':phone' => $data['phone'],
    ':email' => $data['email'],
    ':birth_date' => $data['birthdate'],
    ':gender' => $data['gender'],
    ':biography' => $data['bio']
]);
$appId = $pdo->lastInsertId();
```

Рис. 6 Prepared statements и плейсхолдеры.

Вывод: уязвимость SQL Injection отсутствует.

CSRF (Cross-Site Request Forgery)

Описание уязвимости

CSRF — атака, при которой вредоносный сайт заставляет браузер авторизованного пользователя выполнить нежелательное действие на целевом сайте. Браузер автоматически отправляет cookie при каждом запросе, поэтому сервер не может отличить легитимный запрос от поддельного без дополнительной защиты.

Применённые методы защиты

Реализован паттерн Double Submit Cookie.

При каждом GET-запросе сервер генерирует случайный CSRF-токен, сохраняет его в сессионной cookie (csrf_token) и передаёт в скрытом поле формы (_csrf).

При POST-запросе токен из формы сравнивается с токеном из cookie через hash_equals().

CSRF-защита реализована в файле csrf.php и интегрирована во все формы (form.php, edit.php, login.php, admin.php).

В обработчике POST вызывается validateCSRFToken().

```
$cookieToken = getCookieValue(CSRF_COOKIE_NAME);  
if ($cookieToken === null) return false;  
$formToken = $_POST[CSRF_FIELD_NAME] ?? $_GET[CSRF_FIELD_NAME] ?? '';  
return hash_equals($cookieToken, $formToken);
```

Рис 7. Генерация и проверка csrf токена.

```
<form action="form.php" method="POST">  
  <input type="hidden" name="_csrf" value="<?= $csrfToken ?>">
```

Рис 8. Скрытое поле с csrf.

```
if (!validateCSRFToken()) {  
    http_response_code(403);  
    echo 'Request denied';  
    exit;  
}
```

Рис 9. Проверка csrf при запросе.

Вывод: уязвимость CSRF отсутствует.

Include и Upload уязвимости

Include

Уязвимость характерна для языков с динамическим подключением файлов (например, PHP include()). В данном приложении все подключаемые файлы заданы статически через require_once с фиксированными путями (не зависят от пользовательского ввода). Динамическое подключение по данным из запроса отсутствует. Таким образом, уязвимость типа Include неприменима.

```
session_start();  
require_once __DIR__ . '/db.php';  
require_once __DIR__ . '/config.php';  
require_once __DIR__ . '/cookies.php';  
require_once __DIR__ . '/validation.php';  
require_once __DIR__ . '/db_functions.php';  
require_once __DIR__ . '/auth.php';  
require_once __DIR__ . '/jwt.php';  
require_once __DIR__ . '/csrf.php';
```

Рис 10. Статическое подключение файлов.

Upload

Уязвимость возникает, когда приложение принимает загружаемые файлы без проверки типа и содержимого. В приложении отсутствуют формы загрузки файлов. Все поля принимают только текстовые данные, которые проходят

валидацию регулярными выражениями на сервере. Уязвимость типа Upload неприменима.